```
y for development label]
    end

    subgraph PMEM[PM & EM]
        AddMilestone[Add to milestone]
    end

    Start --> RefinementLabel
    RefinementLabel --> CreateRefIssue
    CreateRefIssue --> DistributeTasks
    DistributeTasks --> RefineIssue
    RefineIssue --> NeedBreakdown
    NeedBreakdown -->|Yes| CreateNewIssues
    CreateNewIssues --> RefineIssue
    NeedBreakdown -->|No| FullyRefined
    FullyRefined --> ReadyLabel
    ReadyLabel --> AddMilestone
    AddMilestone --> End([Development process starts])

    style Start fill:f9f,stroke:333,stroke-width:2px
    style End fill:f9f,stroke:333,stroke-width:2px
    style NeedBreakdown fill:ffd,stroke:333

    classDef pmStyle fill:e6f3ff,stroke:333
    classDef emStyle fill:fff0e6,stroke:333
    classDef engStyle fill:e6ffe6,stroke:333
    classDef sharedStyle fill:f0f0f0,stroke:333

    class RefinementLabel pmStyle
    class CreateRefIssue,DistributeTasks emStyle
    class RefineIssue,NeedBreakdown,CreateNewIssues,FullyRefined,ReadyLabel engStyle
    class AddMilestone sharedStyle
```

## Issue Refinement Checklist

For issues that need refinement, the Engineer/EM should add a comment using this template and complete all checklist items.

If you cannot finish any of these steps, ping EM/PM.

```
plaintext
 Issue Refinement Checklist

 Problem verification
- [ ] Issue label is ~"workflow::refinement"
- [ ] Issue title clearly describes the feature or change
- [ ] Issue description defines requirements and expectations
- [ ] Required permissions and access levels defined
```

Implementation plan

- [ ] A comment with an implementation plan is created
- [ ] Issue is small and doesn't need to be broken down

Final steps
- [ ] This issue has a weight
- [ ] There are no blockers
- [ ] Issue has ~"workflow::ready for development" label

Bug Refinement Checklist

For bug reports that need refinement, the Engineer/EM should add a comment using this template and complete all checklist items.

plaintext
Bug Refinement Checklist

Bug verification
- [ ] Issue label is ~"workflow::refinement"
- [ ] Issue label is ~"type::bug"
- [ ] Issue title clearly describes the bug
- [ ] Steps to reproduce are documented
- [ ] Issue is still reproducible
- [ ] Severity labels are defined
- [ ] Related logs or error messages are attached

Technical analysis
- [ ] Root cause has been identified or hypothesized
- [ ] Affected components/services are identified
- [ ] Potential side effects of the fix are considered

Implementation plan
- [ ] A comment with an implementation plan is created
- [ ] Fix scope is contained and doesn't require larger refactoring
- [ ] Test cases to verify the fix are defined

Final steps
- [ ] This issue has a weight
- [ ] There are no blockers
- [ ] Issue has ~"workflow::ready for development" label

Implementation plan

Add a comment to the issue under refinement using the provided template.

plaintext
Implementation Plan

1. Approach

<!-- Provide a high-level description of the implementation idea -->

2. Dependencies

- [ ] Requires ~backend
- [ ] Requires ~frontend
- [ ] Requires ~database
- [ ] Requires ~documentation
- [ ] Requires ~UX work
- [ ] External service dependencies identified
- [ ] Requires ~API changes

3. Implementation Steps

<!-- Provide step by step description of what needs to be done -->

- Task 1
- Task 2
- Task 3

4. Edge Cases

<!-- Does the implementation cover all scenarios (success, failure) -->

- Success scenarios:
  - Case 1
  - Case 2

- Error scenarios:
  - Case 1
  - Case 2

- Edge conditions:
  - Case 1
  - Case 2

@engineerusername please review this implementation plan.
<!--
Pick a peer engineer following this criteria:
1. is a subject matter expert.
2. might have some familiarity with the topic. or
3. ask on slack who'd be available to review this plan before the due date of the issue
-->

Epics, issues, and tasks

The Source Code team uses the following structure of planning objects to organize work:

1. Epics: are used to identify a larger set of work that aligns to a specific category/theme (most broad) or feature (most specific) that has multiple issues for delivery and spans multiple milestones worth of work.
1. Issues: are used for individual items that will be planned and can be delivered in a single milestone.
1. Tasks: can be created by the issue's DRI inside an issue to further define pieces that need to be delivered as part of completing the issue. Ex: For Pair Programming, For granular details on the progress, etc.

## Convention over configuration

As stated in our direction, we must place special emphasis on our convention over configuration principle. As the feature set within Create:Source Code grows, it may feel natural to solve problems with configuration. To ensure this is not the case, we must intentionally challenge MVC and new feature issues to check for this. Let's consider the following steps for best results:

1. Once issues have been labeled as workflow::needs issue review PM will share the proposal with either a peer or their manager as well as engineering (EM or IC) and product designer.

1. Peers in product and engineering who review the issue should look for opportunities to eliminate configuration where possible. If opportunities are identified, the issue is moved back to workflow::refinement.

1. If PM and peers are satisfied with the proposal and it follows our convention over configuration principle as much as possible, those who reviewed the issue indicate their agreement with the proposal (with either · or a comment in the issue). Finally, PM or EM will label issue workflow:: ready for development.

## Collaborating with counterparts

You are encouraged to work as closely as needed with stable counterparts outside of the PM. We specifically include quality engineering and application security counterparts prior to a release kickoff and as-needed during code reviews or issue concerns.

Quality engineering is included in our workflow via the Quad Planning Process.

Application Security will be involved in our workflow at the same time that kickoff emails are sent to the team, so that they are able to review the upcoming milestone work, and notate any concerns or potential risks that we should be aware of.

## Communication

As a team we strive to be responsive and accommodating when we communicate. When consuming information, we use the following emoji to definition table as a shorthand glossary:

| Emoji | Definition
|

|-------|----------
----------|
| ·  | I'm looking at this now
|
| ·  | I need to think about this before responding
|
| ·  | I don't have time, will come back later
|
| ··  | I don't know
|
| ·  | I understand
|
| ·  | Agree (and often won't give additional comment)
|
| ·  | Task is complete
|
| ·  | I've seen this but I don't think I'm the best person for the job. Ping me if no one
else responds and you need help |

Merge Request reviews

{{% include "includes/engineering/create/conventional-comments.md" %}}

Requesting a review

For an initial review, it's recommended to select a reviewer from the Source Code team.

For maintainer reviews, you can follow the recommendations from the Reviewer Roulette. For time-sensitive or complex reviews, it's preferable to choose a reviewer from the Source Code team.

Triage process

The weekly Triage Report is generated automatically by the GitLab bot and this report is reviewed by the EM. Here is an example of a previous report.

The Triage Report can be quite long, and it important to deal with it efficiently. An effective way to approach it is:

- Open every issue in a separate browser tab and use "edit issue" to mark then as checked once review, then close the tab.
- Verify if the issue belongs to ~"group::source code" and change group label if needed. The Features by Group page is a good starting point for this assessment.
- Apply ~frontend if it is a frontend issue.
- Perform a brief search to assess if is a duplicate, close with a ~Duplicate label if this is the case.
- Is it a ~"support request" ? Does it ~"needs investigation" ? Apply labels if so.
- Apply ~regression label if it is one, consider bumping severity numbers if recent.
- Apply ~"severity::4", ~"priority::4", %Backlog if a smaller issue with no significant impact.
- If an uncontroversial problem with a clear solution, consider applying ~"Seeking community

contributions"
- If also an easier issue which might interest a newer community contributor, consider applying ~"quick win".
- Apply ~"priority::3" ~"severity::3" if a bug with a workaround.
- Anything causing data loss, severe performance impact or security apply a ~"severity::1" and ~"priority::1" or ~"priority::2" and assign to a team member.
- Unassign yourself from the Triage Report

### Engineering cycle

The engineering cycle is centered around the GitLab Release Date every month. This is the only fixed date in the month, and the table below indicates how the other dates can be determined in a given month.

### Iteration documents

These documents comprise everything that is documented during the release planning and execution.

### Issue boards

Create Source Code BE planning takes inputs from the following sources:

- Performance Board
- Infradev Board
- Application Limits Board
- Security Board
- Missed SLO Board
- Bugs
- New features

Create Source Code UX planning takes inputs from the following sources:

- SCM UX Planning Board
- SCM UX Build Board

### Planning issue

Each month a planning issue is created by one of the EMs, using automated tools based on the Source Code issue template.

### Planning board

The Planning Board is created for each release by the PM, and is a curated list of issues by category. The EM requests engineers to assist in refining issues and allocate weights via the refinement process.

### Capacity planning spreadsheet

The EM maintains a Google Sheet for calculating team capacity, and the same Spreadsheet is also

used to perform the process of assigning issues to the release based on weight and priority.

### Build board

The EM selects issues from the Planning Board based on:

- slipped issues
- weight
- priority
- PM preference

The EM then applies the ~Deliverable label to each issue in the Release and assigns then to an engineer. The issues are tracked through the release via the Build Board.

### Candidate issues

Urgent issues are tentatively assigned to a release to ensure other teams have visibility.

At this point the issues are Candidate issues, and the milestone does not confirm that they will be definitely scheduled. Issues move from Candidate status to confirmed during the Issue selection process.

### Key dates

| Date | Event |
| ------ | ------ | ------ |
| The Monday of the week the milestone ends |PM creates planning board and pings EMs in the Planning Issue for review & weighting.<br><br> EMs calculate capacity, add to Planning Issue.<br><br>PM submits RPIs for reviews.|
| Monday to Friday of the week the milestone ends |EMs & ICs add weights to issues in the planning board|
| The Friday the milestone ends | EMs add ~Deliverable labels to issues so that they appear on the Build board as a draft<br><br>Release Post: EMs, PMs, and PDs contribute to MRs for Usability, Performance Improvements, and Bug Fixes|
| The Friday the milestone ends | EMs adjust ~Deliverable labels for slippage and make final assignments<br><br>PMs review final plan for milestone on Build board<br><br>EMs merge RPI MRs for features that have been merged.|
| The third Thursday of the month | Release |

### Weighting issues

We use a system of weights to assist in forecasting the capacity each issue will require to be completed.

These are either assigned by the EM or by engineers ad-hoc or following the refinement process.

### Weight categories

The weights we use are:

{{% include "includes/engineering/create/weighttable.md" %}}

A weight of 5 generally indicates the problem is not clear or a solution should be instead converted to an Epic with sub-issues.

### If the issue should be broken down

If the problem is well-defined but too large (weight 5 or greater), either:

- Promote the issue to an Epic and break the work into sub-issues. Weight the individual issues if possible.
- Ping the EM and PM and outline the reason the issue needed to be promoted to an Epic.

### If the issue SSOT is not clear

- Don't assign a weight, instead add a comment indicating what needs clarification and ping the EM and PM.

### If the issue needs a spike

- Don't assign a weight, instead add a comment about the need for a spike (and possibly what would be investigated) and ping the EM or PM.
- Spikes are scheduled with a weight of 2.
- Spikes are scheduled with a weight of 2 (timeboxed).

See the spike issues section for more details about these issues.

### Security issues

Security issues are typically weighted one level higher than they would normally appear from the table above. This is to account for additional work and backports in the patch release process.

### Planning issue review

The Source Code stable counterparts (BE, FE, PM, UX) meet and propose issues to be worked on in the upcoming release. Using the Mural visual collaboration tool, candidate issues are voted on by the group.

### Capacity planning

Capacity planning is a collaborative effort involving all Source Code team members and stable counterparts from Frontend, UX and Product. An initial list of issues is tracked in the Source Code Group Planning issue example for each month.

### Team availability

Approximately 5-10 business days before the start of a new release, the EM will begin determining how "available" the team will be. Some of the things that will be taken into account when determining availability are:

- Upcoming training
- Upcoming time off / holidays
- Upcoming on-call slots
- Potential time spent on another teams deliverables

Availability is a percentage calculated by (work days available / work days in release)  100.

All individual contributors start with a "weight budget" of 10, meaning they are capable (based on historical data) of completing a maximum number of issues worth 10 weight points total (IE: 2 issues which are weighted at 5 and 5, or 10 issues weighted at 1 each, etc.) Then, based on their availability percentage, weight budgets are reduced individually. For example, if you are 80% available, your weight budget becomes 8.

Product will prioritize issues based on the teams total weight budget. Our planning rotation will help assign weights to issues that product intends on prioritizing, to help gauge the amount of work prioritized versus the amount we can handle prior to a kickoff.

### Source Code issue pipeline

The Source Code issue pipeline is broad, and the PM and EM work together throughout the planning process and the final list is chosen during the Issue Selection meeting. The issue pipeline includes:

- Features
- Bugs
- Security issues
- Infradev board issues
- Performance board issues
- Application limit board issues

### Issue selection

On or around the 16th, the PM and EM meet once more to finalize the list of issues in the release. The issue board for that release is then updated, and any issues with an candidate milestone that are not selected will be moved to Backlog or added as a candidate for a future release.

Issues scheduled for the release are then marked ~"workflow::ready for development".

### Issue assignments

Issue assignments are done collaboratively during the monthly Backlog Refinement meeting and Milestone Planning meetings.
If any priority issues emerge after these meetings, or if assignments can't be done during these meetings, the EM will assign the issues directly, before the milestone starts.

### Follow up issues

You will begin to collect follow-up issues when you've worked on something in a release but have tasks leftover, such as technical debt, feature flag rollouts or removals, or non-blocking

work for the issue. For these, you can address them in at least 2 ways:

- Add an appropriate future milestone to the follow-up issue(s) with a weight and good description on the importance of working this issue
- If a parent issue is fixed but pending activation via a Feature Flag, update the parent issue description to: ·This change has been merged. Rollout will be managed in this rollout issue. Once the Rollout issue is closed then this change will be live.·. Also, link the related Feature Flag issue to the parent issue.
- Add the issue(s) to the relevant planning issue

You should generally take on follow-up work that is part of our definition of done, preferably in the same milestone as the original work, or the one immediately following. If this represents a substantial amount of work, bring it to your manager's attention, as it may affect scheduling decisions.

If there are many follow-up issues, consider creating an epic.

### Spike issues

{{% include "includes/engineering/create/spike-issues.md" %}}

### Double-assign for overly-complex or time-sensitive issues

As discussed in a previous retrospective, in addition to breaking down issues, we should assign two engineers to each task instead of just one for overly-complex or time-sensitive issues. This co-ownership will help parallelize efforts in multiple-MR tasks, speed up immediate code reviews, and ultimately lead to faster delivery of results.

### Backend and Frontend issues

Many issues require work on both the backend and frontend, but the weight of that work may not be the same. Since an issue can only have a single weight set on it, we use scoped labels instead when this is the case: ~backend-weight::<number> and ~frontend-weight::<number>.

### Workflow labels

{{% engineering/workflow-labels group-label="group::source code" %}}

### Retrospectives

We have 1 regularly scheduled "Per Milestone" retrospective, and can have ad-hoc "Per Project" retrospectives.

### Per Milestone

{{% engineering/create-retrospectives group-label="Source Code" group-slug="source-code" use-coordinator="1" %}}

### Per Project

If a particular issue, feature, or other sort of project turns into a particularly useful learning experience, we may hold a synchronous or asynchronous retrospective to learn from it. If you feel like something you're working on deserves a retrospective:

1. Create an issue explaining why you want to have a retrospective and indicate whether this should be synchronous or asynchronous
1. Include your EM and anyone else who should be involved (PM, counterparts, etc)
1. Coordinate a synchronous meeting if applicable

All feedback from the retrospective should ultimately end up in the issue for reference purposes.

Deep dives

{{% include "includes/engineering/create/deep-dives.md" %}}

Career development

{{% engineering/create/career-development "Source Code" %}}

Performance monitoring

The Create:Source Code BE team is responsible for keeping some API endpoints and controller actions performant (e.g. below our target speed index).

Here are some Kibana visualizations that give a quick overview on how they perform:

- Create::Source Code: Controller Actions%2CrefreshInterval%3A(pause%3A!t%2Cvalue%3A0)%2Ctime%3A(from%3Anow-7d%2Cto%3Anow
- Create::Source Code: Endpoints%2CrefreshInterval%3A(pause%3A!t%2Cvalue%3A0)%2Ctime%3A(from%3Anow-7d%2Cto%3An

These tables are filtered by the endpoints and controller actions that the group handles and sorted by P90 (slowest first) for the last 7 days by default.

---
## SECTION: engineering/devops/dev/create/source-code/frontend/_index.md

Common Links

| Category | Handle |
|---|---|
| GitLab Team Handle | Not available |
| Slack Channel | gcreatesource-code-review-fe |
| Slack Handle | Not available |
| Team Boards | Current Milestone |
| Issue Tracker | group::source code + frontend in gitlab-org/gitlab |
| Sentry dashboard | Errors in the last 7 days |

## Team Vision

A central piece in GitLab users' experience, innovating and keeping the experience delightful for all product categories that fall under the Source Code group of the Create stage of the DevOps lifecycle. For information on our product direction, visit the Category Direction - Source Code Management page.

## Team Mission

Support all our counterparts with frontend engineering expertise, including implementation, tech debt management, and timely frontend insights in discovery phases.

## Commonly Monitored Issue Lists

Source Code + Frontend issues
Milestone Planning Issues
Triage reports
Feature flag reports
OKRs (confidential)

## Team Members

The following people are permanent members of the Create:Source Code FE Team:

{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Source Code" team=".Frontend.Create:Source Code" >}}

## Stable Counterparts

The following members of other functional teams are our stable counterparts:

{{< engineering/stable-counterparts role="(Product Manager|Backend Engineer|Technical Writer|Software Engineer in Test|Senior Security Engineer).(Create:Source Code|Create \(Source)|Dev\:Create" >}}

## Core Responsibilities

Collaborate with Product and UX on ideation, refinement and scheduling of relevant work
Provide Frontend support for feature development, bug fixes, under the Source Code Management Product Category
Address bug reports and regressions
Identify and prepare maintenance work to improve developer experience
Collaborate on efforts across the Frontend department

## Projects

### Active Project Table

| Start Date | Project | Description | Tech Lead |
| ------ | ------ | ------ | ------ |

| 2023-09 | New Diffs (Epic) | A project to deliver a reusable and performant way of rendering diffs across GitLab | · |
| 2024-10 | Directory and File Page Improvements | A project to improve user experience of header area for directory and file pages | · |

Archived Project Table

| Start Date | End Date |Project | Description | Tech Lead |
| ------ | ------ | ------ | ------ | ------ |
| 2023 | put on hold | Blame info in Blob page | Improve usability of repository by rendering blame information in blob page | · |
| 2023 | 2024 | Branch Rules - Edit | Allow editing the branch rule details in one place | · |
| 2022-09 | 2023-04 | Branch Rules - Overview | Place all settings pertaining to branch rules in one place - overview only | · |
| 2021 | 2022 | Refactor Repository browser into 1 vue app | Render the blob page within the Repository frontend app for smoother experience | · |

Engineering Onboarding

Work

In general, we use the standard GitLab engineering workflow. To get in touch with the Create:Source Code FE team, it's best to create an issue in the relevant project (typically GitLab) and add the ~"group::source code" and ~frontend labels, along with any other appropriate labels (~devops::create, ~section::dev). Then, feel free to ping the relevant Product Manager and/or
Engineering Manager as listed above.

For more urgent items, feel free to use gcreatesourcecode or gcreatesourcecodefe on Slack.

Take a look at the features we support per category here.

Code Reviewing

To prevent the creation of knowledge silos and also receiving input from people outside of the team, we follow these principles:

Not all Merge Requests need to go through the team
However, Merge Requests that seem important for the team to be aware, let's ensure one of the reviews go through a team member

MRs important to the team: these are changes to logic in our apps or meaningful component changes. Sequential work in a larger epic is also beneficial to have oversight from peers within the team. But bottom line, use your best judgement.

{{% include "includes/engineering/create/conventional-comments.md" %}}

Issue refinement

1. Once we have validated the problem, product, UX, and engineering will collaborate to propose

a solution and decide on what's technically feasible. The proposed solution may be shared with users to validate it solves the problem.

    1. Issues that require design work are marked with UX and workflow::ready for design.
    1. Issues in the design process are marked with workflow::design.
    1. Once designs are ready and the proposed solution is viable then the label workflow::planning breakdown will be applied.

1. Once we have confirmed the proposed solution is viable, we will move to break it down as much as possible. When issues are ready for this stage, PM will mark issues with workflow::refinement label to signal next step.
1. We have 1 regularly scheduled "Per Milestone" backlong refinement session. During then, we review the issues in the backlog together. Each engineer also proposes issues they believe are valuable for the team to work on as deliverables candidates.

    1. If an issue needs further refinement, we'll distribute tasks labeled workflow::refinement among engineers.
    1. If an issue does not need further refinement, the label workflow::ready for development will be applied.
    1. If the team believes the issue should be worked on in the next milestone, the milestone will be set. Otherwise, the label scm-backlog will be applied and the issue will be placed in %backlog milestone.

1. EM will create a refinement issue (example) and distribute tasks labeled workflow::refinement among engineers.
1. Engineers or EM will follow the checklist for assigned issues, work with PM, UX, and other engineering counterparts where necessary to address questions and concerns.
1. If the planned implementation of the issue can be further broken down, the engineer/EM will work with the PM to reduce scope and create new issues until this is the case (either PM or engineer/EM can create new work items).
1. Once an issue is fully refined, engineers or EM will add an appropriate weight and label it as workflow::ready for development. These issues can then be added to the milestone.

Diagram

mermaid
flowchart TD
    A[Problem Validation] --> B[Solution Collaboration]
    B --> C{Design Required?}
    C -->|Yes| D[Mark with UX and workflow::ready for design]
    D --> E[Issue in design process with workflow::design]
    E --> F[Designs ready]
    C -->|No| F
    F --> G[Mark with workflow::planning breakdown]
    G --> H[Solution Viable?]
    H -->|Yes| I[Mark with workflow::refinement]
    I --> J[Per Milestone Backlog Refinement Session]
    J --> K{Needs Further Refinement?}
    K -->|Yes| L[EM creates refinement issue]
    L --> M[Engineers assigned to refinement tasks]
    M --> N[Engineers follow checklist]
    N --> O{Can be broken down further?}
    O -->|Yes| P[Reduce scope and create new issues]
    P --> N

```
    O -->|No| Q[Add weight and mark workflow::ready for development]
    K -->|No| Q
    Q --> R{Work in next milestone?}
    R -->|Yes| S[Set milestone]
    R -->|No| T[Apply scm-backlog label and place in %backlog milestone]

    %% Adding colors to different stages
    style A fill:d4f1f9,stroke:05a,stroke-width:2px
    style B fill:d4f1f9,stroke:05a,stroke-width:2px
    style C fill:ffe6cc,stroke:d79b00,stroke-width:2px
    style D fill:e1d5e7,stroke:9673a6,stroke-width:2px
    style E fill:e1d5e7,stroke:9673a6,stroke-width:2px
    style F fill:e1d5e7,stroke:9673a6,stroke-width:2px
    style G fill:d5e8d4,stroke:82b366,stroke-width:2px
    style H fill:ffe6cc,stroke:d79b00,stroke-width:2px
    style I fill:d5e8d4,stroke:82b366,stroke-width:2px
    style J fill:d5e8d4,stroke:82b366,stroke-width:2px
    style K fill:ffe6cc,stroke:d79b00,stroke-width:2px
    style L fill:fff2cc,stroke:d6b656,stroke-width:2px
    style M fill:fff2cc,stroke:d6b656,stroke-width:2px
    style N fill:fff2cc,stroke:d6b656,stroke-width:2px
    style O fill:ffe6cc,stroke:d79b00,stroke-width:2px
    style P fill:fff2cc,stroke:d6b656,stroke-width:2px
    style Q fill:d5e8d4,stroke:82b366,stroke-width:2px
    style R fill:ffe6cc,stroke:d79b00,stroke-width:2px
    style S fill:f8cecc,stroke:b85450,stroke-width:2px
    style T fill:f8cecc,stroke:b85450,stroke-width:2px
```

## Issue Refinement Checklist

For issues that need refinement, the Engineer/EM should add a comment using this template and complete all checklist items.

If you cannot finish any of these steps, ping EM/PM.

plaintext

```
## Issue Refinement Checklist

### Problem verification
- [ ] Issue label is ~"workflow::refinement"
- [ ] Issue title clearly describes the feature or change
- [ ] Issue description defines requirements and expectations
- [ ] Required permissions and access levels defined
- [ ] Desktop design is defined (if needed)
- [ ] Mobile design is defined (if needed)

### Implementation plan
```

- [ ] A comment with an implementation plan is created
- [ ] Implementation plan includes accessibility specification (if needed)
- [ ] Implementation plan includes proposal for ~"analytics instrumentation" (if needed)
- [ ] A separate issue is opened for adding ~"analytics instrumentation" and linked to this issue (if needed)
- [ ] Implementation plan includes ~documentation changes (if needed)
- [ ] Implementation plan includes proposal for adding feature tests (if needed)
- [ ] Implementation plan includes feature tests (if needed)
- [ ] Issue is small and doesn't need to be broken down

Final steps
- [ ] This issue has a weight
- [ ] There are no blockers
- [ ] Issue has ~"workflow::ready for development" label

Bug Refinement Checklist

For bug reports that need refinement, the Engineer/EM should add a comment using this template and complete all checklist items.

plaintext
Bug Refinement Checklist

Bug verification
- [ ] Issue label is ~"workflow::refinement"
- [ ] Issue label is ~"type::bug"
- [ ] Issue title clearly describes the bug
- [ ] Steps to reproduce are documented
- [ ] Issue is still reproducible
- [ ] Severity labels are defined
- [ ] Related logs or error messages are attached

Technical analysis
- [ ] Root cause has been identified or hypothesized
- [ ] Affected components/services are identified
- [ ] Potential side effects of the fix are considered

Implementation plan
- [ ] A comment with an implementation plan is created
- [ ] Fix scope is contained and doesn't require larger refactoring
- [ ] Test cases to verify the fix are defined

Final steps
- [ ] This issue has a weight
- [ ] There are no blockers
- [ ] Issue has ~"workflow::ready for development" label

Implementation plan

Add a comment to the issue under refinement using the provided template.

plaintext
Implementation Plan

1. Approach

<!-- Provide a high-level description of the implementation idea -->

2. Dependencies

- [ ] Requires ~backend
- [ ] Requires ~frontend
- [ ] Requires ~database
- [ ] Requires ~documentation
- [ ] Requires ~UX work
- [ ] External service dependencies identified
- [ ] Requires ~API changes

3. Implementation Steps

<!-- Provide step by step description of what needs to be done -->

- Task 1
- Task 2
- Task 3

4. Edge Cases

<!-- Does the implementation cover all scenarios (success, failure) -->

- Success scenarios:
  - Case 1
  - Case 2

- Error scenarios:
  - Case 1
  - Case 2

- Edge conditions:
  - Case 1
  - Case 2

@engineerusername please review this implementation plan.
<!--
Pick a peer engineer following this criteria:
1. is a subject matter expert.
2. might have some familiarity with the topic. or
3. ask on slack who'd be available to review this plan before the due date of the issue

-->

### Capacity planning

{{% include "includes/engineering/create/capacity-planning-fe.md" %}}

### Weights

{{% include "includes/engineering/create/weights-fe.md" %}}

### Example of Weights

w1: Blame view - "authored" line leaking into next row

w2: CSV rendering hangs viewer for large files

w3: Edit Branch Rules: Update selector to support searching Deploy Keys

### Source code context

When weighing issues that have to do with Blob view, make sure to take into account the duality of Blob. We use both HAML and Vue to render the Blob view. There is a high chance that you will implement your changes for both. Majority of file types uses Vue architecture. Though there are some file types that need backend syntax highlighter and therefore are rendered with HAML. The same will happen, when an error occurs.

This duality will be resolved with the Link to package managers when viewing dependency files with highlight.js.

### Spike issues

{{% include "includes/engineering/create/spike-issues.md" %}}

### Workflow labels

{{% engineering/workflow-labels group-label="group::source code" %}}

### Async standup

The groups in the Source Code group conduct asynchronous standups in the gcreatesourcecodestandup channel every Monday.

The goal is to support the members of these groups in connecting at a personal level, not to check in on people·s progress or replace any existing processes to communicate status or ask for help, and the questions are written with that in mind:

What did you do outside of work since we last spoke?
What are you planning to do today?
Is anything blocking your progress or productivity?

For more background, see the Async standup feedback issue on the Create stage issue tracker.

### Retrospectives

We have 1 regularly scheduled "Per Milestone" retrospective, and can have ad-hoc "Per Feature" retrospectives more focused at analyzing a specific case, usually looking into the Iteration approach.

### Per Milestone

{{% engineering/create-retrospectives group-label="Source Code" group-slug="source-code" use-coordinator="1" %}}

### Milestone Kickoff & Retrospective review

At the start of each milestone we have a synchronous Kickoff session where every IC take turns at presenting their plan for their Deliverables for the new milestone.

This happens at least 2 working days after all Deliverables are assigned, which happens on the first day of the milestone.

If any additional information is needed from design, a list of requests will be issued to the designer after the meeting.

During this call, we also do a quick Retrospective review going through the highlights of the discussions in the asynchronous issue mentioned above.

### Other related pages

#### Issues

April 2020: Frontend: Iteration Retrospective (Source Code)

---
## SECTION: engineering/devops/dev/create/talent-assessments/index.md

# Talent Assessments

For company-wide guidance on Talent Assessments, please consult the People Group's Talent Assessment page.

## Performance assessment

As described in the main Talent Assessment > Performance Factor > Measuring Performance section, the managers look at several Performance Indicators holistically to assess in which Performance box to place a team member.

Let's go over which indicators are taken into account during Talent Assessments to add clarity and establish expectations.

Performance Indicators

These are the indicators managers might consider when assessing Performance. Not one in isolation, but interconnecting all of them.

Some of these are tracked through Tableau dashboards while others are assessed manually by the manager.

MR Rate
Reviews Rate
Business Results
Number of projects as Maintainer
Number of interviews performed
Mentoring activities
Security issues fixed/contributed to
InfraDev work
Discretionary Bonuses
TMRG/DIB Initiatives participation
...
(to be completed)

Note: This list is but a start, we aim to complete this and add detail over time.

---
## SECTION: engineering/devops/dev/create/tech-lead/index.md

What is a Tech Lead

This role is outlined here.

Create Tech Leads

Source Code Tech Leads

| Start Date | End Date | Estimated End Date | Epic(s) / Issue(s) | Tech Lead |
|---|---|---|---|---|
| 2024-02-01 | TBD | TBD | Branch Rules | Joe Woodward |
| 2024-02-01 | TBD | TBD | Cells 1.0 | Vasilii Iakliushin |

Code Review Tech Leads

| Start Date | End Date | Estimated End Date | Epic(s) / Issue(s) | Tech Lead |
|---|---|---|---|---|
| 2024-01-30 | TBD | TBD | Cells 1.0 | Kerri Miller |
| 2024-02-13 | TBD | TBD | New Diffs | Stanislav Lashmanov |
| 2024-02-19 | TBD | TBD | Review Rounds | Phil Hughes |

Remote Development Tech Leads

| Start Date | End Date | Estimated End Date | Epic(s) / Issue(s) | Tech Lead |
|---|---|---|---|---|
| 2025-03-24 | TBD | TBD | Web IDE Category | Enrique Alcántara |
| 2024-05-01 | TBD | TBD | Workspaces Category | Chad Woolley |

How is a Tech Lead different from a Domain Expert

Tech Leads and Domain Experts share similarities and differences. The tables below clarify distinctions between the two roles.

Similarities

| Criteria | Tech Lead | Domain Expert |
| --- | --- | --- |
| Expertise Requirement | Requires domain knowledge and expertise | Requires substantial experience in a specific area |
| Collaboration | Collaboration with Engineering Managers and Product Managers | Collaboration with team members and stakeholders |
| Mentoring | Provides technical guidance and mentoring | Mentors team members in their specific area |
| Not a Managerial Role | Not a manager | Not a manager |

Differences

| Criteria | Tech Lead | Domain Expert |
| --- | --- | --- |
| Nature of Role | Temporary role tied to a specific topic/project | Ongoing role with substantial expertise |
| Assignment Criteria | Efficiency, domain knowledge, expertise | Substantial experience in specific areas |
| Seniority | Any engineer, regardless of seniority | Encouraged for team members with experience |
| Providing Technical Vision | Providing technical vision and architecture | Not required. |
| Guidance and Mentoring | Offering technical guidance and mentoring | Conducting code and MR reviews in their domain |
| Planning and Prioritizing Work | Planning and prioritizing work | Not required |
| Tracking Progress and Reporting | Tracking progress on commitments and reporting | Ensuring progress tracking and reporting within their domain, providing insights into achievements, challenges, and goals |
| Risk Management | Identifying, assessing, and managing technical risks that may impact deliverables | Managing technical risks associated with specific technology, product feature, or codebase area |
| Coordination | Overseeing the work of others and helping remove blockers | |

Coordinating with team members in their domain, ensuring smooth collaboration and addressing challenges as they arise |
| Collaboration | Slack channel for collaboration (techleads or topic/project specific channel) | Not required. |
| Project Template | Utilizes a project template for guidance | Not required. |
| Scope | Tied to a specific topic/project | Generally encompasses specific technology, feature, or codebase area |

Responsibilities

The Tech Lead responsibilites are outlined here.

Process

Determine if the Project needs a Tech Lead

In some projects, having a Tech Lead depends on the team's available capacity. Engineering Managers decide based on the team's Tech Leadership capacity. Use these questions to help evaluate if your project needs a Tech Lead.

1. Is the project technically complex? Does it require a blueprint?
1. Does it involve multiple technologies or integrations?
1. Does the project demand unique technical expertise?
1. Does the project have a high level of risk associated with technical aspects?
1. Is the success of the project critical to the overall business goals?
1. Are there complex technical decisions to be made during the project?
1. Does the project involve architectural choices that need experienced guidance?
1. Is there a tight deadline for project completion?
1. Is there a need for effective communication between technical and non-technical team members?

Appointing a Tech Lead for a Project

| Step | Engineering Manager's Responsibilities |
|---|---|
| 1 | Assess project needs |
| 2 | Evaluate Tech Leadership capacity |
| 3 | Select a Tech Lead |

---
## SECTION: engineering/devops/dev/plan/_index.md

Plan teams:

- Plan:Project Management Team
- Plan:Product Planning Team
- Plan:Knowledge Team

The responsibilities of this collective team are described by the Plan stage. Among other things, this means
working on GitLab's functionality around issues, boards, milestones, to-do list, issue lists and filtering, roadmaps, time tracking, requirements management, notifications, value stream analytics (VSA), wiki, and pages.

- I have a question. Who do I ask?

In GitLab issues, questions should start by @ mentioning the Product Manager for the corresponding Plan stage group. GitLab team-members can also use splan.

For UX questions, @ mention the Product Designers on the Plan stage; Nick Leonard for Plan:Project Management, Nick Brandt for Plan:Product Planning, and Libor Vanc for Plan:Optimize. Plan:Knowledge should follow the process for groups without a designer.

## How we work

- In accordance with our GitLab values.
- Transparently: nearly everything is public, we record/livestream meetings whenever possible.
- We get a chance to work on the things we want to work on.
- Everyone can contribute; no silos.
- We do an optional, asynchronous daily stand-up in splanstandup.

## Workflow

We work in a continuous Kanban manner while still aligning with Milestones and GitLab's Product Development Flow.

## Capacity Planning

When we're planning capacity for a future release, we consider the following:

1. Availability of the teams during the next release. (Whether people are out of the office, or have other demands on their time coming up.)
1. Work that is currently in development but not finished.
1. Historical delivery (by weight) per group.

The first item gives us a comparison to our maximum capacity. For instance, if the team has four people, and one of them is taking half the month off, then we can say the team has 87.5% (7/8) of its maximum capacity.

The second item is challenging and it's easy to understimate how much work is left on a given issue once it's been started, particularly if that issue is blocking other issues. We don't currently re-weight issues that carry over (to preserve the original weight), so this is fairly vague at present.

The third item tells us how we've been doing previously. If the trend is downwards, we can look to discuss this in our retrospectives.

Subtracting the carry over weight (item 2) from our expected capacity (the product of items 1

and 3) should tell us our capacity for the next release.

### Estimating effort

Groups within Plan use the same numerical scale when estimating upcoming work.

{{% include "includes/engineering/plan/estimating-effort.md" %}}

### Issues

Issues have the following lifecycle. The colored circles above each workflow stage represents the emphasis we place on collaborating across the entire lifecycle of an issue; and that disciplines will naturally have differing levels of effort required dependent upon where the issue is in the process. If you have suggestions for improving this illustration, you can leave comments directly on the whimsical diagram.

Everyone is encouraged to move issues to different workflows if they feel they belong somewhere else. In order to keep issues constantly refined, when moving an issue to a different workflow stage, please review any open discussions within the issue and update the description with any decisions that have been made. This ensures that descriptions are laid out clearly, keeping with our value of Transparency.

### Epics

If an issue is > 3 weight, it should be promoted to an epic (quick action) and split it up into multiple issues. It's helpful to add a task list with each task representing a vertical feature slice (MVC) on the newly promoted Epic. This enables us to practice "Just In Time Planning" by creating new issues from the task list as there is space downstream for implementation. When creating new vertical feature slices from an epic, please remember to add the appropriate labels - devops::plan, group::, Category: or feature label, and the appropriate workflow stage label - and attach all of the stories that represent the larger epic. This will help capture the larger effort on the roadmap and make it easier to schedule.

### Design Documents

For all tier T1 and T2 roadmap items, and initiatives spanning multiple milestones, we recommend creating
a design document using the
Architecture design workflow.
This approach offers several benefits:

1. Single Source of Truth (SSOT): A design document serves as the central place for all important information related
   to the initiative, reducing time spent searching for decisions across various places.
2. Increased Visibility: By creating design documents, we raise awareness of the work done in the
   Plan stage, such as the work items framework,
   customizable Work Item Types, custom fields, custom status,

GLQL, frontend-driven views, and many more.
3. Discoverability: Design documents are easily accessible
through our public handbook,
aligning with engineering best practices.
4. Collaborative Decision-Making: Changes and discussions occur through merge requests,
ensuring visibility to all involved team members.
5. Comprehensive Entry Point: The design document functions as a primary entry point for the initiative, containing:
- An executive summary
- Links to related epics, issues, and wiki pages
- Links to Status updates
- Implementation details
- A decision log or embedded decisions within the document
- Links to relevant boards or dashboards

This comprehensive approach allows easy onboarding for team members and provides stakeholders with all necessary information in one place.

This is the recommended workflow for all initiatives:

1. Create a Slack channel with the convention f[feature name].
2. Develop a design document using the Architecture evolution workflow.
Get started using this template.
You don't need to fill out all sections. This is a living document and it's expected that it evolves over time.
3. An epic hierarchy is created with sub-epics mapping to iterations, each achievable within a milestone.
4. Iterations are broken into multiple issues that can be accomplished independently, and PMs schedule those as normal.
5. Other actions may be established, such as regular 'office hours' calls.

Team members should collaborate to continuously refine the iterations and update the design document as complexity is revealed. This approach ensures that all stakeholders have a clear, up-to-date understanding of the initiatives's progress and implementation details.

Roadmap

In product development at GitLab, Product is responsible for the what and why, Engineering is responsible for the how and when [1]. Maintaining a credible roadmap is therefore a collaborative process, requiring input from both.

The Product Roadmap outlines what the team aims to accomplish over a 4-6 quarter timeline. It is shared across the organization to ensure alignment with the go-to-market strategy and enable reliable commitments to customers.

Changes to the Plan Product Roadmap, made by the Product Manager, are reviewed and accepted by the Engineering Manager of the affected group. This happens at least once a month and is captured in a Wiki Page.

Most items being reviewed during roadmap planning have not yet had detailed technical

investigation from engineering. Planning at this resolution is intended to be thoughtful but not perfect. Velocity remains our priority.

## Reviewing the Roadmap

By performing a review, Engineering Managers play a key role by ensuring the roadmap is achievable and effectively sequenced to maximize velocity. Below are some best practices to guide a thoughtful review:

- Assess Achievability: Is the timeline realistic given the team's current capacity, skills, and dependencies?
- Account for Technical Preparation: Does the roadmap allocate time for necessary technical preparation, such as technical spikes or investigations?
- Optimize Team Utilization: Does the sequence of work align with the team·s skill profile, avoiding periods of underutilization or skill mismatches?
- Evaluate Redundancy: How robust is the rest of the roadmap if one item takes longer than anticipated?
- Clarify Requirements: Do you sufficiently understand each proposed change or do you need additional information?
- Ensure Shared Understanding: Do you and your Product and UX counterparts have a shared understanding of all terminology used?
- Seek Opportunities to Optimize: Have you identified opportunities to iterate or increase velocity by adjusting the order of work?
- Reduce Friction: Is the sequence of work likely to cause avoidable conflicts, such as multiple engineers committing to the same codebase areas simultaneously?
- Identify Process-Driven Delays: Are there items expected to take longer due to process requirements (e.g., multi-version compatibility) rather than capacity constraints?
- Account for Cross-Team Dependencies: Are there cross-team dependencies that could put parts of the timeline at risk?
- Incorporate a Buffer: Is a proportion of capacity allowed for exogenous shocks; such as unexpected PTO, or a high-severity incident?
- Lean on Your Experience: When you look at the roadmap as a whole and think about recent quarters, does it look achievable?

## Roadmap Organization

```mermaid
graph TD;
  A["devops::plan"] --> B["group::"];
  B --> C["Category:"];
  B --> D["non-category feature"];
  C --> E["maturity::minimal"];
  C --> F["maturity::viable"];
  C --> G["maturity::complete"];
  C --> H["maturity::lovable"];
  E--> I["Iterative Epic(s)"];
  F--> I;
  G --> I;
  H --> I;
  D --> I;
```

```
I--> J["Issues"];
```

## Executing on the Roadmap

Every Roadmap commitment has a Directly Responsible Individual (DRI) for its delivery, which is typically a Tech Lead who leads project management activities such as clarifying scope, coordinating dependencies, and communicating progress. If no engineer in the group has the capacity to assume a Tech Lead role, the Engineering Manager (EM) may step in. The EM is ultimately accountable for overall roadmap execution and cross-team coordination in either case.

The project manager clarifies scope, identifies dependent work, appoints DRIs for work streams, and ensures risks and blockers are prioritized.

The DRI maintains a Wiki page or design document for the project containing a project timeline, project status, links to work items, key participants, a dogfooding proposal, and a decision register. This is encouraged for all important projects, especially Tier 1 and Tier 2 Roadmap commitments. It acts as a Single Source of Truth (SSoT) that greatly improves cross-functional collaboration and ensures decisions made are captured. Previous examples are:

- Configurable Statuses
- Custom Fields
- Issue Work Item Type
- Epic Work Item Type

## Internal Testing

Plan Engineering regularly tests new functionality internally before releasing to customers. As part of a drive to improve quality in the work we deliver to customers, this process is divided into two parts.

### Alpha Testing

Testing that occurs during ongoing development. This is limited to GitLab's subgroups or projects other than gitlab-org, gitlab-com, or gitlab-org/gitlab. The Plan Stage has two groups that are available for testing on: gl-demo-ultimate-plan-stage and gitlab-org/plan-stage.

### End-of-line testing

End-of-line (EOL) testing is the final step before release to customers. The finished product is delivered to all GitLab team-members, usually by enabling it for the gitlab-com and gitlab-org groups. This is accompanied by collection of internal feedback, typically using a feedback issue. The minimum duration of this period of testing is determined by the Engineering Manager.

No new scope will be accepted at this time without significant justification and without restarting the testing period. Only defects and fit & finish issues identified during testing will be addressed.

This practice ensures that, while there may be more than one item in end-of-line testing at the

same time, the system under test resembles as closely as possible the one intended to be given to customers.

### Dogfooding

Dogfooding helps to build confidence in feature readiness and identify shortcomings before they reach the customer. In most cases, if an improvement cannot be adopted for a useful workflow internally it should not be expected to land with customers either. Identifying a dogfooding opportunity ahead of time can help to reach consensus on what the minimum valuable change should include.

Dogfooding opportunities should be meaningful rather than hypothetical. A new workflow is adopted, an existing workflow complemented or improved, or made redundant.

Project leads should strive to implement dogfooding during the final testing phase and should expect to observe some adoption.

### Talking With Customers

We aim to have cross-functional representation in every conversation we have with customers.

### Customer Conversations calendar

Anyone who is scheduling a call with a customer via sales, conducting usability reasearch, or generally setting up a time to speak with customers or prospects is encouraged to add the Plan Customer Conversations calender as an invitee to the event. This will automatically populate the shared calendar with upcoming customer and user iteractions. All team members are welcome and encouraged to join -- even if it's just to listen in and get context.

You can subscribe to the calendar and invite it as a participant in a customer meeting that you are scheduling using the URL gitlab.com5icpbg534ot25ujlo58hr05jd0@group.calendar.google.com.

### Shadow a customer call

All team members are welcome and encouraged to join customer calls -- even if it's just to listen in and get context.

To ensure upcoming calls appear in your calendar, subscribe to the Plan Customer Conversations calendar. Product Managers add upcoming customer interviews to this calendar and you're welcome to shadow any call.

1. In GCal, next to "Other Calendars" in the left sidebar, click the +
1. Select "Subscribe to Calendar"
1. In the "Add Calendar" input, paste gitlab.com5icpbg534ot25ujlo58hr05jd0@group.calendar.google.com

Upcoming customer calls will often be advertised in the splan channel in advance, so look out there also.

### Review previous calls

All recorded customer calls, with consent of the customer, are made available for Plan team-members to view in Dovetail.

To access these, simply go to the Plan Customer Calls project on Dovetail and log in with Google SSO. More information is available in the Readme of this project.

If you find you do not have access, reach out to a Plan PM and ask to be added as a Viewer.

### Review previous UX Research calls

UX Research calls are scripted calls designed to mitigate bias and to address specific questions related to user needs and/or usability of the product. A selection of UX Research calls are available in the Plan Customer Calls Dovetail Project in the column titled UXR - Research and Validation.

### Board Refinement

We perform many board refinement tasks asynchronously, using GitLab issues in the Plan project. The policies for these issues are defined in triage-ops/policies/plan-stage. A full list of refinement issues is available by filtering by the ~"Plan stage refinement" label.

### Tracking Committed Work for an Upcoming Release

While we operate in a continuous Kanban manner, we want to be able to report on and communicate if an issue or epic is on track to be completed by a Milestone's due date. To provide insight and clarity on status we will leverage Issue/Epic Health Status on priority issues.

### Keeping Health Status Accurate

At the beginning of the Milestone, Deliverable issues will automatically be updated to "On Track". As the Milestone progresses, assignees should update Health Status as appropriate to surface risk or concerns as quickly as possible, and to jumpstart collaboration on getting an issue back to "On Track".

At specific points through the milestone the Health Status will be automatically degraded if the issue fails to progress. Assignees can override this setting any time if they disagree. The policy that manages this automation is here. It can be disabled for any individual issue by adding the ~"Untrack Health Status" label.

### Health Status Definitions for Plan

- On Track - We are confident this issue will be completed and live for the current milestone
- Needs Attention - There are concerns, new complexity, or unanswered questions that if left unattended will result in the issue missing its targeted release. Collaboration needed to get back On Track
- At Risk - The issue in its current state will not make the planned release and immediate action is needed to rectify the situation

### Flagging Risk is not a Negative

We feel it is important to document and communicate, that changing of any item's Health Status to "Needs Attention" or "At Risk" is not a negative action or something to be cause anxiety or concern. Raising risk early helps the team to respond and resolve problems faster and should be encouraged.

## OKRs

### Active Quarter OKRs

FY25-Q2 Stage-level Objectives are available here (internal).

### Previous Quarter OKRs

FY25-Q1 Stage-level Objectives all closed out between 74% and 88% and are available here (internal).

### Drafting OKRs using GitLab

Guidance is available, including a video guide, on Approach to OKRs at GitLab.
GitLab currently offers some freedom in how to structure OKR hierarchies. We take the following approach in Plan:

- EMs are encouraged to create group-level KRs under stage-level Objectives directly, without creating their own OKR structure.
- Group KRs and Stage Objectives should ladder into a higher Objective, which can exist anywhere in the organization. In the development of OKRs a stage-level Objective laddered directly into a CEO KR.
- They should be created or added as child objectives and key results of their parent so that progress roll-ups are visible.
- Product development goals are established in milestone planning, following the regular Product Development Flow, and not in OKRs.

Doing this ensures the hierarchy will be as simple, consistent and shallow as possible. This improves navigability and visibility, as we currently don't have good hierarchy visualization for OKRs.

An example of a valid single OKR hierarchy is:

```
mermaid
flowchart TD
    A[Plan Objective] --> B(Project Management KR)
    A --> C[Product Planning KR]
    A --> D[Optimize KR]
    A --> E[Knowledge KR]
    A --> K[Principal Engineer KR]
    A --> L[SEM KR]
```

Ownership is indicated using labels and assignee(s). The label indicates the group and/or stage, assignee the DRI.

OKRs should have the following labels:

- Group, Stage, and Section (as appropriate).
- Division (~"Division::Engineering") to distinguish from other functions.
- updates::[weekly, semi-monthly, monthly] depending on how often the OKR is expected to be updated by the DRI.

## Retrospectives

The Plan stage conducts monthly retrospectives asynchronously using GitLab issues. Monthly retrospectives are performed in a Confidential Issue made Public upon Close. Confidentiality of these Issues while Open aligns with GitLab SAFE Framework.

Where necessary, the use of Internal Notes is encouraged to further adhere to SAFE Guidelines. Internal notes remain confidential to participants of the retrospective even after the issue is made public, including Guest users of the parent group.

Examples of information that should remain Confidential per SAFE guidelines are any company confidential information that is not public, any data that reveals information not generally known or not available externally which may be considered sensitive information, and material non-public information.

Retrospective issues are created by a scheduled pipeline in the async-retrospectives project. They are then updated once the milestone is complete with shipped and missed deliverables. For more information on how it works, see that project's README.

Each EM is the DRI for conducting and concluding their group's retrospective, along with summary and corrective actions.

The role of the DRI is to facilitate a psychologically safe environment where team-members feel empowered to give feedback with candour. As such they should refrain from participating directly. Instead they should publicise, conclude and make improvements to the retrospective process itself.

## Timeline

- 27th (Previous Month) A retrospective issue is automatically created for the milestone in progress.
- 18th The milestone is closed and open issues in the build phase are labeled with ~"missed deliverable".
- 21st The issue description is automatically updated with shipped and missed deliverables and the team are tagged to add feedback.
- 4th (Next Month) A final reminder is created automatically in splan for final feedback.
- 5th (Next Month) The DRI concludes the retrospective.

## Dogfooding Value Stream Analytics (VSA) in the Milestone Retrospective

To improve the retrospective data-driven experience, we are dogfooding VSA to simplify the data collection for the retrospective. This been done by automatically adding a link to the VSA of

the current milestone filtered by group/stage to the retrospective.
With Value stream analytics (VSA) our team is getting visibility to the lifecycle metrics of each milestone through the breakdown of the end-to-end workflow into stages. This allows us to identify bottlenecks and take action to optimize actual flow of work.

For example, for the review phase, we are using VSA to count the time between ·Merge request reviewer first assigned" to ·Merge request last approved at·.. With this data, we can identify:

- MRs that were bottlenecked due to limited reviewers/maintainers capacity.
- Slow review start times & Idle time post-approval.
- MRs with multiple feedback loops.
- Whether long review time originates from same-team MR reviews or out-of-team MR reviews.

Please leave your feedback in this issue.

Concluding the Retrospective

The DRI is responsible for completing the following actions:

- Adding a comment to the retrospective issue summarizing actionable discussion items and suggesting corrective actions.
- Finding a DRI for each corrective action. Creating an issue in gl-retrospectives/plan for each is optional, but doing so and adding the ~"follow-up" label will ensure they're included automatically in the next retrospective.
- Recording a short summary video and sharing in splan. This can be discussed in the next weekly team call and can be added to the Plan Stage playlist on Youtube so that it shows up on team pages.
- Closing the issue and making it public.

In both the summary comment and video the DRI should be particularly careful to ensure all information disclosed is SAFE. If the retrospective discussion contains examples of unSAFE information, the issue should not be made public.

Regressions

Regressions contribute to the impression that the product is brittle and unreliable. They are a form of waste, requiring the original (lost) effort to be compounded further with a fix or a reversion and reimplementation of the intended behavior.

Engineering Managers are strongly encouraged to conduct a simple Root Cause Analysis (RCA) when a regression takes place in a feature owned by their group, in order to:

- Inform the author and reviewers of the original MR that it caused a regression.
- Define corrective actions that might prevent or reduce the likelihood of a similar regression in future.
- Identify trends or patterns that can lead to human error.

The following RCA format was trialed in a FY23 Q2 OKR. It can be posted as a comment on the original MR when the regression has been successfully reverted.